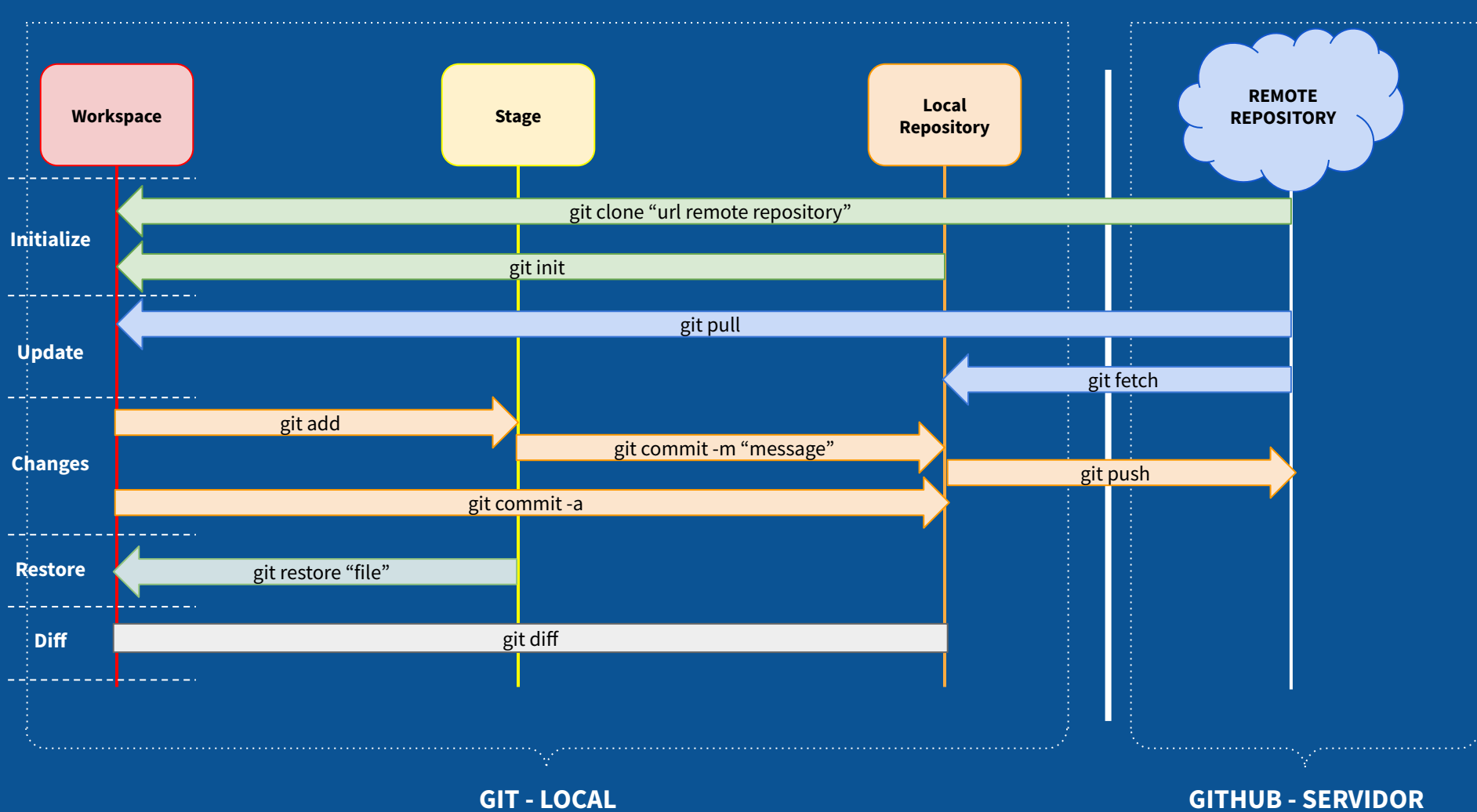


RESUM GIT I GITHUB

CICLE FORMATIU DE GRAU SUPERIOR
DESENVOLUPAMENT D'APLICACIONS MULTIPLATAFORMA / WEB

Autor: Roger Sobrino Gil



SERVIDOR

GITHUB - <https://github.com/>

**REMOTE
REPOSITORY**

Pàgina web on es guarda el codi del projecte. Té altres funcions, però ara ens centrarem en guardar el codi del projecte i, així, poder compartir-lo amb altres integrants.

Per copiar el repositori (projecte) del servidor al PC es fa servir la comanda (des de la terminal del PC):

\$git clone “url repository remot”

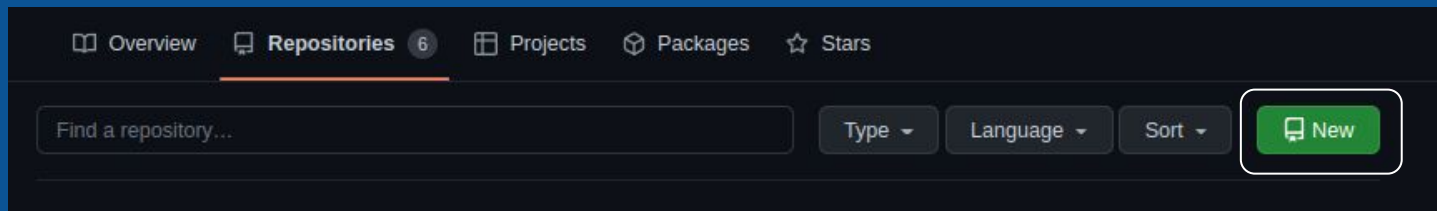
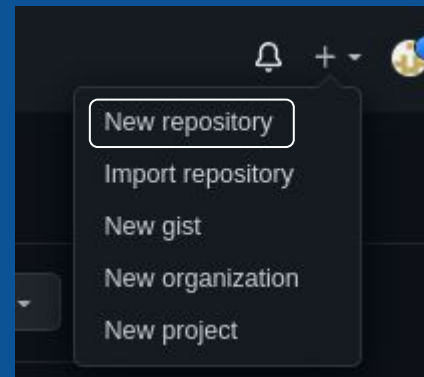
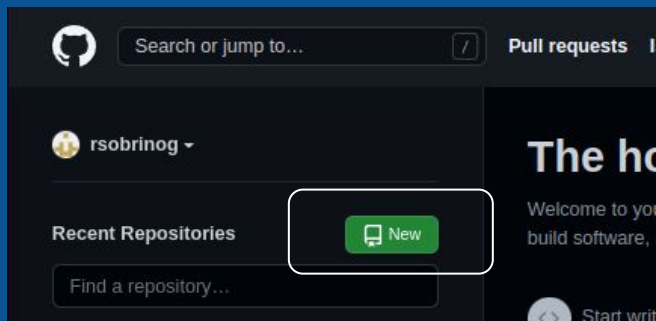
Si s'utilitza git clone, NO s'utilitza git init i vicerversa

SERVIDOR



GITHUB - Repositori/Projecte

Hi ha varies maneres de crear un repositori:



SERVIDOR



GITHUB - Repositori/Projecte

Create a new repository

A repository contains all project files, including the revision history. Already have a project repository elsewhere? [Import a repository.](#)

Owner * / Repository name *

 rsobrinog /

Great repository names are short and memorable. Need inspiration? How about [super-octo-rotary-phone](#)?

Description (optional)

☒ **Public**
Anyone on the internet can see this repository. You choose who can commit.

☐ **Private**
You choose who can see and commit to this repository.

Initialize this repository with:

Skip this step if you're importing an existing repository.

☒ **Add a README file**
This is where you can write a long description for your project. [Learn more.](#)

Add .gitignore

Choose which files not to track from a list of templates. [Learn more.](#)

.gitignore template: None

Choose a license

A license tells others what they can and can't do with your code. [Learn more.](#)

License: None

 You are creating a public repository in your personal account.

Create repository

Per crear el repositori nou sortirà la següent pàgina a emplenar:

Repository name: Nom del projecte a crear.

Description: Descripció breu del projecte.

Vista projecte: Pública o Privada. S'aconsella utilitzar l'opció public per fer proves.

Add a README file: Clicar l'opció per a que es crei el projecte amb un fitxer nou.

Clicar **Create Repository**.

SERVIDOR

GITHUB - Vista principal Repositori



Una vegada creat el repositori tindreu una imatge semblant a la següent en la vista principal del projecte.

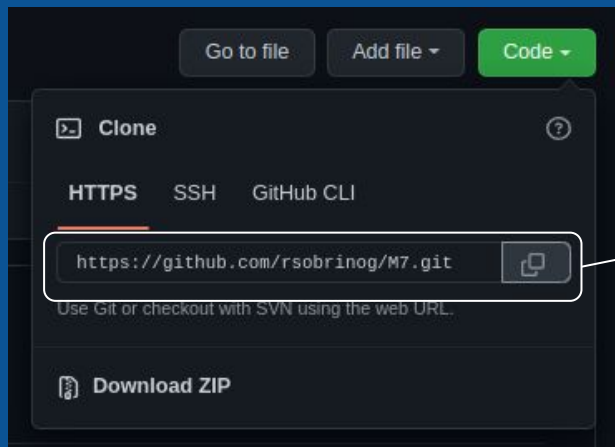
The screenshot shows the GitHub interface for a repository named 'rsobrinog / M7'. At the top, there's a search bar and navigation links like 'Pull requests', 'Issues', 'Marketplace', and 'Explore'. Below this, the repository name 'rsobrinog / M7' is displayed with a 'Public' badge. To the right of the name are buttons for 'Unpin', 'Unwatch' (with a count of 1), 'Fork' (with a count of 0), and 'Star' (with a count of 0). A secondary navigation bar includes links for 'Code', 'Issues', 'Pull requests', 'Actions', 'Projects', 'Wiki', 'Security', 'Insights', and 'Settings'. The main content area shows the 'main' branch with 1 branch and 0 tags. It lists an 'Initial commit' by 'rsobrinog' with a commit hash 'bd0ebf' from 14 days ago, and a file 'README.md' also committed 14 days ago. A preview of the README.md file is shown below, containing the text 'M7'. On the right side, there's an 'About' section with a note that no description, website, or topics are provided, followed by statistics for 'Readme', 'stars' (0), 'watching' (1), and 'forks' (0). At the bottom right, a 'Releases' section indicates 'No releases published' and provides a link to 'Create a new release'.

SERVIDOR



GITHUB - Vista principal Repositori

El següent pas és crear un clone en local del repositori creat al servidor.



url a copiar per afegir a la comanda **\$ git clone**.

LOCAL

GIT - instal·lar git [enllaç](#)

Linux:

Obrir la terminal i posar la següent comanda:

```
$ sudo apt install git-all
```

Per més opciones... ⇒

MacOs:

Obrir la terminal i posar la següent comanda:

```
$ git --version
```

Per més opciones... ⇒

Windows:

Anar al següent [enllaç](#) i seguir els passos indicats.

LOCAL

GIT - Sistema de Control de Versions

Workspace

El següent pas és crear un clone en local del repositori creat al servidor.

REMOTE
REPOSITORY

git clone "url remote repository"

Utilitzant **\$ git clone "url"** es crea un **workspace** en local (PC). Es crearà una copia exacta del repositori/projecte del servidor en local. Això ens permet començar a modificar/crear arxius del repositori.

Dintre de la carpeta del projecte (ubicada al Workspace) podreu trobar un arxiu de nom **.git**. Aquest arxiu indica que la carpeta actual està reservada per un repositori i no s'hi pot clonar cap repositori nou.

LOCAL

GIT - Començant a modificar...

Workspace



Al modificar/crear un nou arxiu dintre del Workspace, git comença a treballar dintre del seu procés.

Una vegada acabada la modificació de l'arxiu o dels arxius es passarà a afegir a la secció **stage** amb la comanda **\$ git add "file"**. En cas de ser més d'un arxiu es fa servir la comanda **\$ git add .**

Workspace



git add

Stage

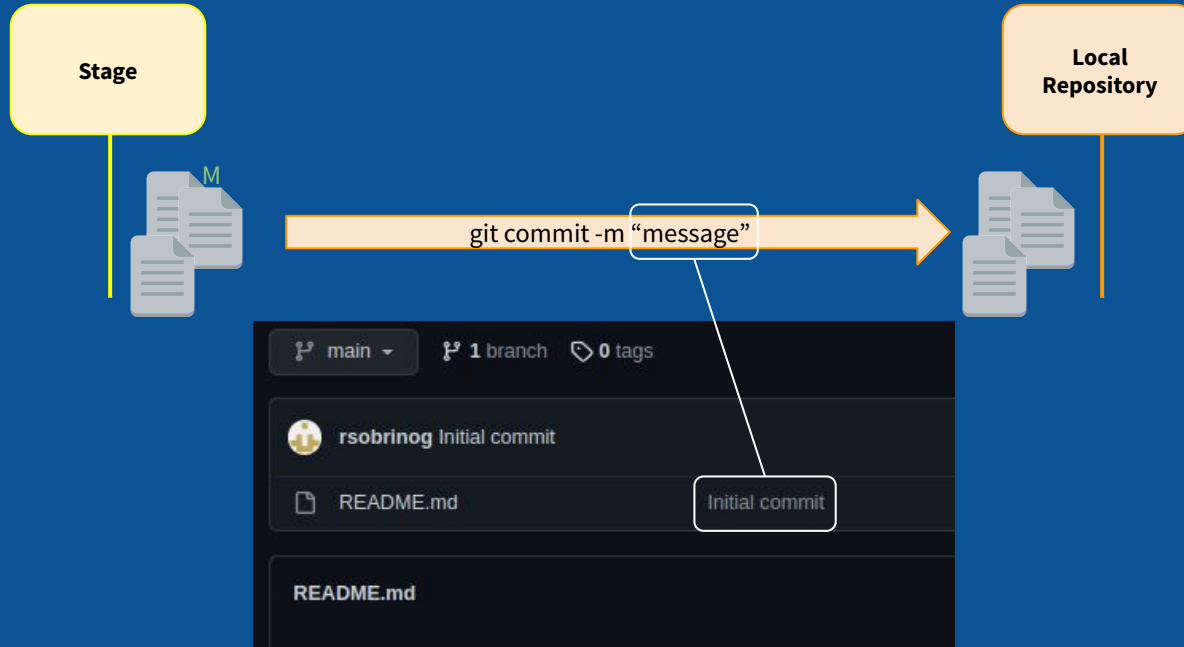


```
roger@rogerMatebook:~/IJB/IJB$ git status -s
M README.md
roger@rogerMatebook:~/IJB/IJB$ git add README.md
roger@rogerMatebook:~/IJB/IJB$ git status -s
M README.md
```

LOCAL

GIT - Crear una versió dels canvis

Quan ja es tenen tots els arxius acabats per crear una versió per guardar, s'utilitza la comanda **\$ git commit -m "message"**.



LOCAL

GIT - Abans de pujar, configurar...

Un pas abans d'utilitzar push i pujar el còdi a github, caldrà que configureu que sou a l'hora de pujar el codi.

Cal utilitzar les següents comandes:

```
$ git config user.email "xxxxxx@jaumebalmes.net"
```

```
$ git config user.name "xxxxxx"
```

Aquestes comandes ajuden a saber qui ha pujat què.

LOCAL

GIT - Pujar la nova versió al servidor

Quan ja es vol pujar la còpia dels arxius modificats i preparats al servidor s'utilitza la comanda **\$ git push**.



LOCAL

ERRORS

Si esteu en windows, en el moment d'utilitzar la comanda **\$ git push** us pot començar a demanar credencials (username i password) . En aquest cas (i si teniu el repositori en public) heu de posar la següent comanda:

\$ git config credential.helper store

\$ git push "url remote repository"

Username: <type your username>

Password: <type your password>

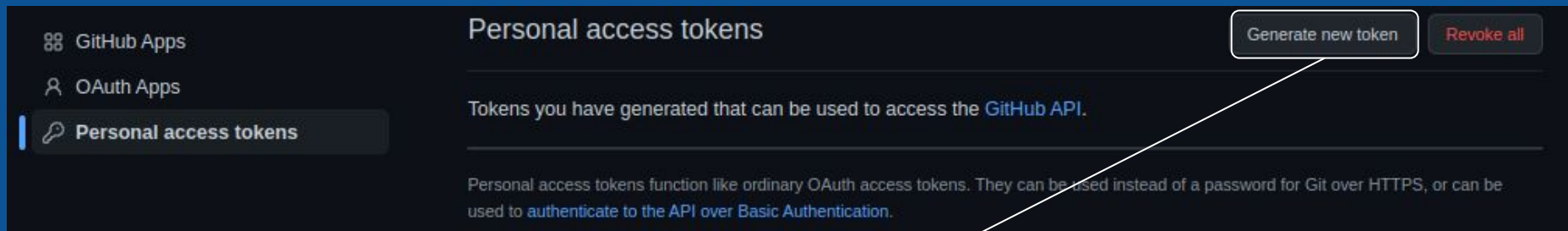
Aquest helper redueix el número de vegades que s'haurà de posar el username o el password.

A la següent diapositiva es veu com extreure el password.

LOCAL

ERRORS

Cal crear el password del repositori creant un token. S'ha d'anar a **settings ⇒ Developer settings ⇒ Personal access tokens**.



Apretar el botó **Generate new token** i posar un nom a **note**, seleccionar els dies que durarà el token a **expiration** i seleccionar la secció **repo**.

Una vegada us surti el token, posar-lo a password. NO perdre mai aquest token.

TREBALLA COL·LABORATIU EN GITHUB

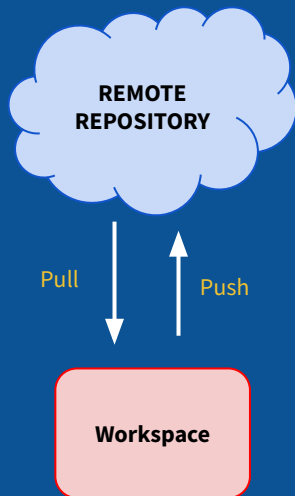
BRANCH

BRANCH

Branques

L'ús de branques és essencial en treballs col·laboratius dintre de Github. Quan fem treballs NO col·laboratius, es treballa en el format que hem vist fins ara. El **cicle del treball col·laboratiu**, s'enten que comença després de fer un clone del repositori.

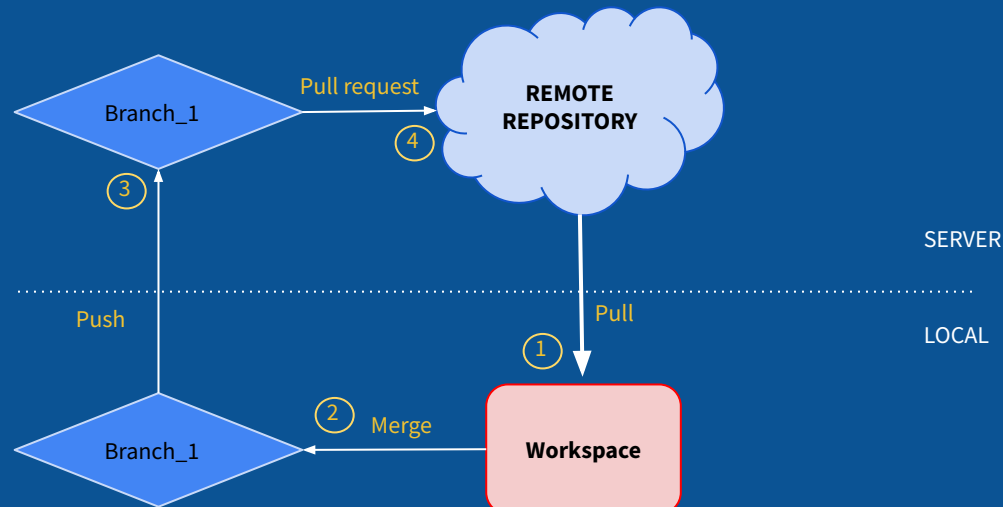
Treball NO col·laboratiu



NO

SI

Treball col·laboratiu



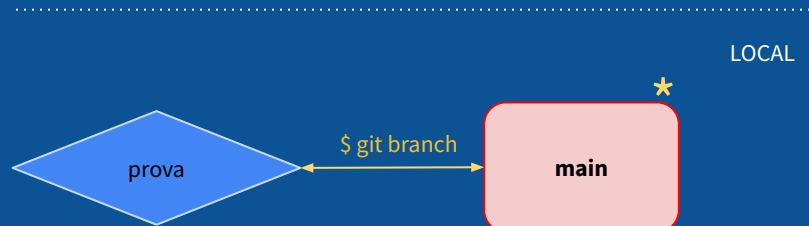
BRANCH

Branques - Crear branca en local

Per treballar de forma correcte en un projecte col·laboratiu cal treballar amb branques (**Branch**). Cal situar-se en la imatge en què ja hem clonat el repositori i, abans de començar a modificar/crear arxius nous, caldrà crear una o varies branques per realitzar les nostres modificacions/creacions.

Per crear una branca s'utilitza la comanda **\$ git branch nombranca**. Una vegada creada la branca, per mirar les branques que tenim s'utilitza la comanda **\$ git branch**.

```
roger@rogerMatebook:~/IJB/IJB$ git branch
* main
  prova
```



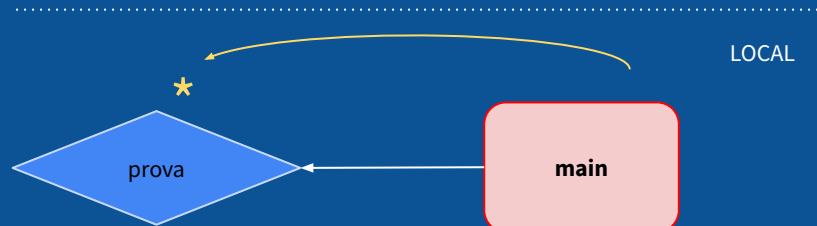
BRANCH

Branques - Moure's entre branques

Per canviar de branques s'utilitza la comanda **\$ git checkout nombranca**.

L'asterisc ens indica a quina branca estem actualment.

```
roger@rogerMatebook:~/IJB/IJB$ git branch
* main
  prova
roger@rogerMatebook:~/IJB/IJB$ git checkout prova
Switched to branch 'prova'
Your branch is up to date with 'origin/prova'.
roger@rogerMatebook:~/IJB/IJB$ git branch
  main
* prova
```



BRANCH

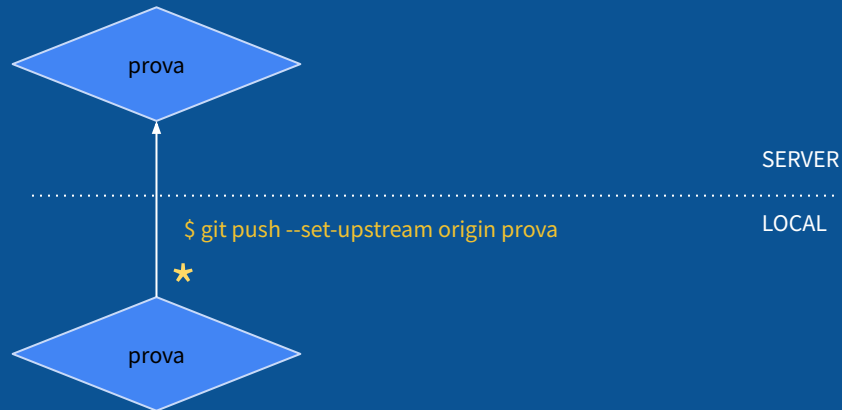
Branques - Actualitzar branca local en servidor

```
roger@rogerMatebook:~/IJB/IJB$ git push --set-upstream origin prova
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0
remote:
remote: Create a pull request for 'prova' on GitHub by visiting:
remote:   https://github.com/rsobrinog/IJB/pull/new/prova
remote:
To https://github.com/rsobrinog/IJB.git
 * [new branch]      prova -> prova
Branch 'prova' set up to track remote branch 'prova' from 'origin'.
```

Quan es crea una branca en local, cal pujar-la al servidor per poder treballar de forma col·laborativa amb la comanda:

\$ git push --set-upstream origin nombranca.

Aquesta comanda es realitza des de la branca nova creada.



BRANCH

Branques - Modificar/crear arxiu/s...

Des de la branca creada (nova) es comencen a modificar/crear arxius. Una vegada es tingui la versió per pujar al servidor, s'utilitzen les comandes:

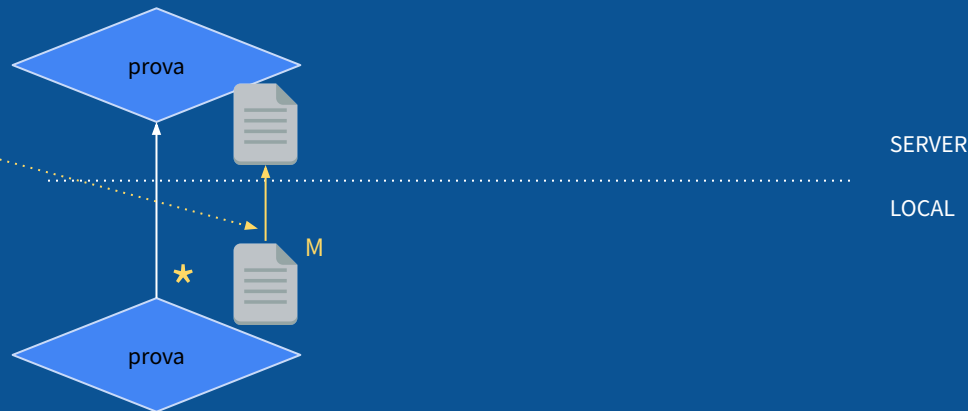
\$ git add

\$ git commit -m "message"

\$ git push

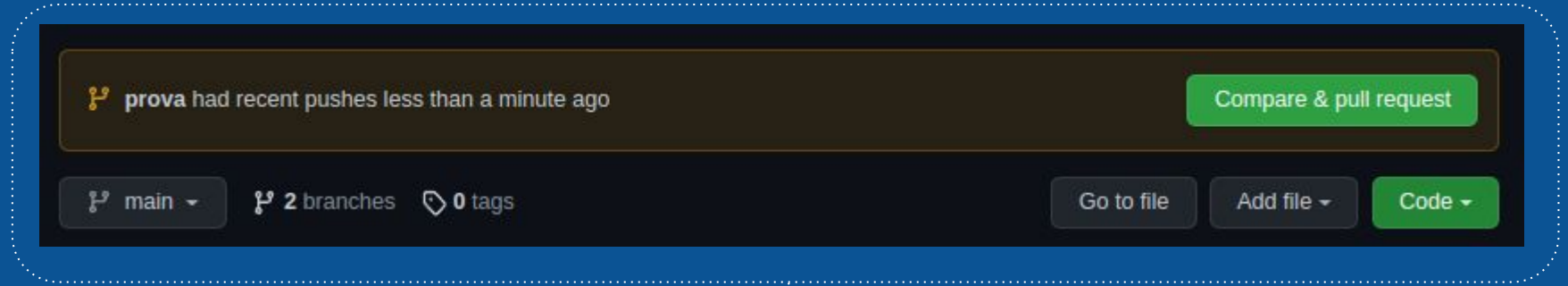
Amb aquestes ordres, aconseguim pujar l'arxiu modificat a la branca del servidor.

```
roger@rogerMatebook:~/IJB/IJB$ git push
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 8 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 311 bytes | 311.00 KiB/s, done.
Total 3 (delta 1), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/rsobrinog/IJB.git
  20f9760..76e8244  prova -> prova
```

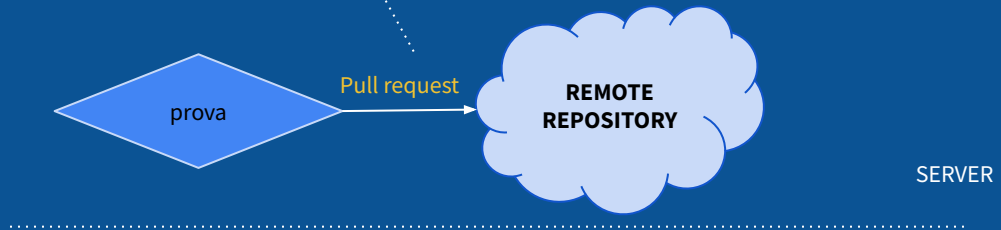


BRANCH

Branques -Confirmar canvis d'una branca a la branca principal en servidor



Per acceptar els canvis d'una branca a la principal cal fer-ho des del servidor (Github)

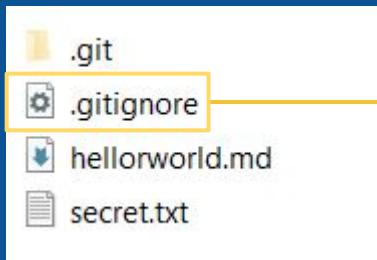


IGNORAR AQUELLS FITXERS QUE NO VOLEM TENIR DINS DEL
CONTROL DE VERSIONS NI PUJAR A GITHUB.

IGNORE

IGNORE

Ignore - Fitxer on definim els fitxers que no s'afegiran a Git.



És un fitxer de configuració de Git on ens permet definir aquells fitxers o carpetes que no s'inclouran en el control de versions de Git i, conseqüentment, no es pujaran a Github.

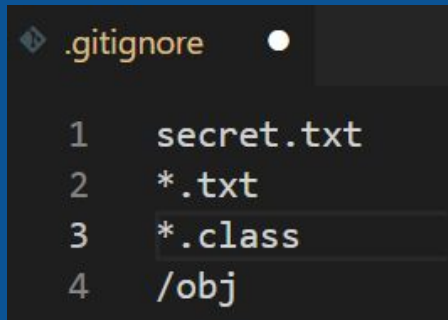
Hem de crear un fitxer que es dirà *.gitignore* al root del projecte. És a dir, a la mateixa ruta on es troba la carpeta *.git* del projecte.

Hi ha altres estratègies per utilitzar el gitignore de forma més genèrica però en aquesta presentació ho enfocarem a nivell de projecte.

IGNORE

Ignore - Definir les regles dins del fitxer que Git ignorarà

Podem utilitzar un editor per modificar el fitxer `.gitignore` i a dins podem definir les regles que Git tindrà en compte per ignorar els fitxers o carpetes.



```
.gitignore
1 secret.txt
2 *.txt
3 *.class
4 /obj
```

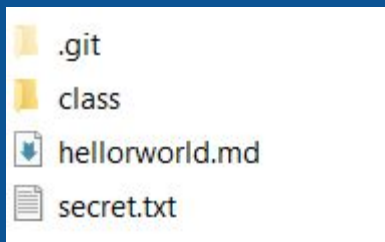
Uns exemples podrien ser:

1. No s'inclourà el fitxer amb nom: *secret.txt*
2. No s'inclourà cap fitxer *.txt*
3. No s'inclourà cap fitxer *.class*
4. No s'inclourà tot allò que està dins la carpeta *obj*.

IGNORE

Ignore - Funcionament

El següent projecte no té el fitxer *.gitignore*



git status

```
On branch master  
  
No commits yet  
  
Untracked files:  
  (use "git add <file>..." to include in what will be committed)  
    class/  
    hellorworld.md  
    secret.txt  
  
nothing added to commit but untracked files present (use "git add" to track)
```

Apareixen tots aquells fitxers i carpetes que s'han modificat en el workspace en la màquina local.

IGNORE

Ignore - Funcionament

En el mateix projecte creem el fitxer *.gitignore* i li diem que no tingui en compte el fitxer *secret.txt*



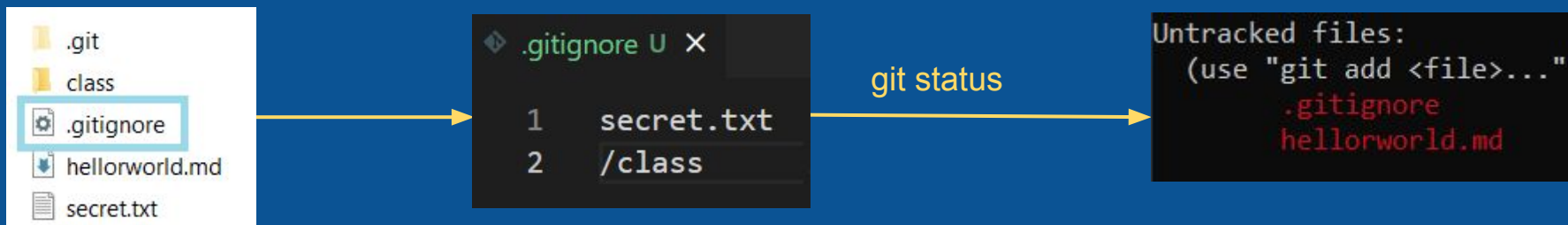
Apareix un fitxer nou: *.gitignore* (ja que és un fitxer que haurem de tenir en compte al projecte i s'haurà de pujar a github)

No apareix el fitxer *secret.txt* (l'ignora i no apareix al fer *git status*)

IGNORE

Ignore - Funcionament

En el mateix projecte creem el fitxer `.gitignore` i li diem que no tingui en compte el fitxer `secret.txt` i la carpeta `class`



Apareix un fitxer nou: `.gitignore` (ja que és un fitxer que haurem de tenir en compte al projecte i s'haurà de pujar a github)

No apareix el fitxer `secret.txt` (l'ignora i no apareix al fer git status)

No apareix la carpeta `class` (l'ignora i no apareix ni la carpeta ni els fitxers fills al fer git status)

IGNORE

Ignore - Funcionament

Es recomana crear el fitxer `.gitignore` i les seves regles a l'inici de crear el projecte.

En el cas de voler ignorar un/s fitxer/s o una carpeta/es és molt important afegir la/es regla/es en el `.gitignore` abans de fer un `git add` (stage) o un `git commit` (repositori local) del projecte.

En cas contrari, si el/s fitxer/s o la/es carpeta/es es troben ja en stage o s'ha fet un commit al repositori local, el fitxer `.gitignore` no funcionarà.

Per a que `.gitignore` funcioni amb els fitxers que tenim a stage o que ja s'ha fet un commit haurem de seguir els següents passos:

1. `git rm -r --cached .` (el punt final és important)
2. modifiquem el fitxer `.gitignore` i hi afegim les regles que volem aplicar
3. `git add .` (el punt final és important)
4. `git commit -m "fixed untracked files"`

MOLTES GRÀCIES!

